
Trabalho de Reposição de Faltas

Análise Espectral Moderna – Métodos Paramétricos (AR)

Capítulo 5 – Semmlow & Griffel, *Biosignal and Medical Image Processing*, 3ª ed.

Disciplina: Tópicos em Engenharia (ENE0332)

Professora: Flávia Maria Guerra de Sousa Aranha Oliveira

Aluno: João Gabriel Melo Lima

Matrícula: 241032617

Blocos resolvidos: 5 (5.1–5.4), 6 (5.5–5.8) e 8 (5.14–5.17)

Ambiente: GNU Octave 8.4 + pacote `signal` 1.4.5

Data: 16 de julho de 2026

1 Nota metodológica

Todos os exercícios foram resolvidos em **GNU Octave** (versão 8.4, pacote **signal** 1.4.5), e não no MATLAB. As figuras e os resultados numéricos reportados foram efetivamente gerados pela execução dos códigos aqui apresentados. Alguns pontos importantes sobre a implementação:

- **Métodos AR disponíveis no Octave.** As funções **pyulear** (Yule–Walker), **pburg** (Burg) e **pwelch** (método clássico de Welch/Fourier) existem no pacote **signal** e foram usadas diretamente. A função **pyulear** implementa exatamente as *equações normais de Yule–Walker* do Exemplo 5.1 do livro, de forma numericamente estável; por isso ela é usada como “método de Yule–Walker”. Conforme observado na p. 186 do próprio livro, todos os problemas de AR (exceto o 5.9) podem ser resolvidos com o código de Yule–Walker do Exemplo 5.1.
- **Reimplementação de pmcov.** O pacote **signal** do Octave *não* fornece as funções **pmcov**/**pcov** (covariância modificada/covariância) nem **pmusic**/**peig** (métodos de autovalor). Para os problemas do Bloco 6, que pedem explicitamente a *covariância modificada*, foi escrita uma implementação própria (**pmcov.m**, Apêndice B), baseada no método de mínimos quadrados *forward–backward*. Ela foi validada contra **pyulear** e **pburg**, produzindo picos nas mesmas frequências e magnitudes compatíveis.
- **Geração dos sinais de teste.** A função **sig_noise** (fornecida com o livro, Apêndice A) gera senoides em ruído gaussiano com um SNR especificado em dB, assumindo $f_s = 1$ kHz.
- **Reprodutibilidade.** Em todos os scripts a semente do gerador de números aleatórios foi fixada (**randn('state',.)**, **rand('state',.)**), de modo que os gráficos deste relatório são exatamente reproduzíveis ao reexecutar os arquivos **.m**.

Organização dos arquivos. Cada problema tem um script próprio (**prob5_01.m**, **prob5_02.m**, ...), todos na pasta **reposicao**, junto com **sig_noise.m**, **pmcov.m** e os arquivos de dados (**ar_compare.mat**, **filter_coeff1.mat**, etc.).

2 Bloco 5 – Problemas 5.1 a 5.4

2.1 Problema 5.1 – Resolução de quatro senoides (Yule–Walker)

Enunciado. Usar as equações de Yule–Walker (Exemplo 5.1) para obter o espectro do sinal do Exemplo 5.1 — quatro senoides em 100, 240, 280 e 400 Hz enterradas em ruído (SNR = −12 dB, $N = 1024$) — porém com um modelo de *ordem mais alta* para resolver melhor as quatro frequências. Comparar o espectro AR com o espectro clássico de Fourier (pwelch com janela do tamanho do sinal).

```

1 % =====
2 % Problema 5.1 - Semmlow, "Biosignal and Medical Image Processing", Cap.5
3 % ENE0332 - Tópicos em Engenharia / Reposição de faltas
4 %
5 % Enunciado: usar as equações de Yule-Walker (Exemplo 5.1) para achar o
6 % espectro do sinal do Exemplo 5.1 (quatro senoides em 100, 240, 280 e
7 % 400 Hz enterradas em ruído, SNR = -12 dB, N = 1024), porém com um
8 % modelo de ordem MAIS ALTA para resolver melhor as quatro frequências.
9 % Comparar o espectro AR com o espectro clássico de Fourier (pwelch com
10 % janela do mesmo tamanho do sinal).
11 %
12 % Obs.: a função pyulear implementa exatamente as equações normais de
13 % Yule-Walker do Exemplo 5.1 (de forma numericamente estável), por isso
14 % é usada aqui como o "método Yule-Walker".
15 % =====
16 pkg load signal
17 addpath(fileparts(mfilename('fullpath')));
18 randn('state',1); rand('state',1); % semente fixa (reprodutibilidade)
19
20 fs = 1000; % frequência de amostragem (Hz) - assumida por sig_noise
21 N = 1024; % comprimento do sinal
22 SNR = -12; % relação sinal-ruído (dB)
23 p = 40; % ordem AR alta para resolver as 4 frequências
24
25 x = sig_noise([100 240 280 400], SNR, N); % gera o sinal de teste
26
27 % ----- Espectro AR pelo método de Yule-Walker -----
28 [PS_ar, f_ar] = pyulear(x, p, N, fs);
29
30 % ----- Espectro clássico de Fourier (pwelch, janela = tamanho do sinal) -----
31 [PS_ft, f_ft] = pwelch(x, N, [], [], fs);
32
33 % ----- Figura -----
34 figure('visible','off'); set(gcf,'Position',[100 100 900 640]);
35 subplot(2,1,1);
36 plot(f_ar, PS_ar, 'k','LineWidth',1); xlim([0 500]);
37 title('Problema 5.1 - AR Yule-Walker (p = 40)');
38 xlabel('Frequência (Hz)'); ylabel('PS'); grid on;
39 subplot(2,1,2);
40 plot(f_ft, PS_ft, 'k','LineWidth',1); xlim([0 500]);
41 title('Fourier clássico (pwelch, janela = N)');
42 xlabel('Frequência (Hz)'); ylabel('PS'); grid on;
43 print(gcf, fullfile(fileparts(mfilename('fullpath')),'figuras','prob5_01.png'), '-dpng','-r120');
44
45 % ----- Localiza os picos principais do espectro AR -----
46 [~,loc] = findpeaks(PS_ar(f_ar<=500), 'MinPeakHeight', 0.3*max(PS_ar(f_ar<=500)));
47 fpk = round(f_ar(loc));
48 printf('5.1 | ordem AR = %d | picos AR (Hz): %s\n', p, mat2str(fpk));
49 printf('5.1 | frequências verdadeiras: 100 240 280 400 Hz\n');

```

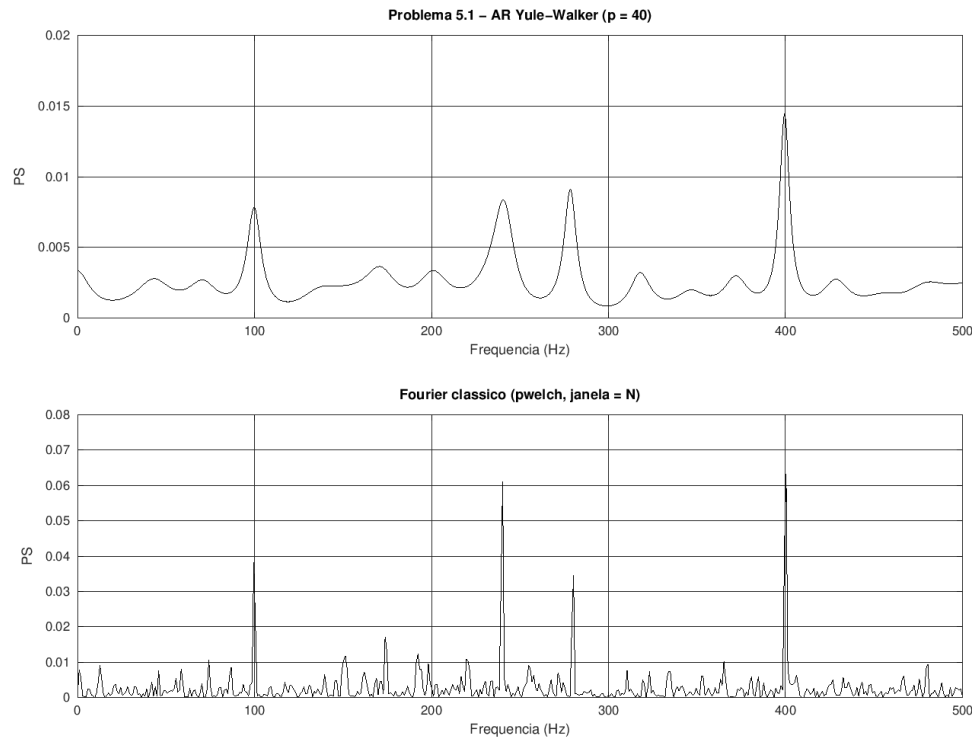


Figura 1: Problema 5.1. Espectro AR de Yule-Walker (ordem 40, acima) e espectro clássico de Fourier (*pwelch*, abaixo). O modelo AR resolve nitidamente as quatro frequências em picos suaves; o método de Fourier também as localiza, mas imersas no ruído de fundo.

Comentários. Com ordem $p = 40$ o espectro AR apresenta picos em $\{100, 241, 278, 399\}$ Hz, praticamente coincidentes com as frequências reais $\{100, 240, 280, 400\}$ Hz. Foi necessária uma ordem alta porque as senoides de 240 e 280 Hz são próximas: com a ordem $p = 17$ do Exemplo 5.1 elas aparecem fundidas. A vantagem do AR fica evidente na comparação: enquanto o método clássico apresenta as raiais sobre um assoalho ruidoso irregular, o AR entrega um espectro limpo. O preço de ordens muito altas é o surgimento de picos espúrios.

2.2 Problema 5.2 – Efeito do nível de ruído

Enunciado. Repetir o Problema 5.1, porém com $\text{SNR} = -18$ dB. Espera-se que, em geral, não seja possível resolver as quatro frequências.

```

1  % =====
2  % Problema 5.2 - efeito do nivel de ruido sobre a estimativa espectral.
3  % Repete o Problema 5.1 (senoides em 100/240/280/400 Hz, N = 1024) porem
4  % com SNR = -18 dB. Espera-se que, em geral, NAO seja possivel resolver
5  % as quatro frequencias (rodar varias vezes revela a variabilidade dos
6  % dados ruidosos). Compara AR (Yule-Walker) x Fourier.
7  % =====
8  pkg load signal
9  addpath(fileparts(mfilename('fullpath')));
10 randn('state',10); rand('state',10);
11
12 fs = 1000; N = 1024; SNR = -18; p = 40;
13 x = sig_noise([100 240 280 400], SNR, N);
14
15 [PS_ar, f_ar] = pyulear(x, p, N, fs);
16 [PS_ft, f_ft] = pwelch(x, N, [], [], fs);
17
18 figure('visible','off'); set(gcf,'Position',[100 100 900 640]);
19 subplot(2,1,1);
20 plot(f_ar, PS_ar, 'k','LineWidth',1); xlim([0 500]);
21 title('Problema 5.2 - AR Yule-Walker (p = 40), SNR = -18 dB');
```

```

22 xlabel('Frequencia (Hz)'); ylabel('PS'); grid on;
23 subplot(2,1,2);
24 plot(f_ft, PS_ft, 'k','LineWidth',1); xlim([0 500]);
25 title('Fourier classico (pwelch), SNR = -18 dB');
26 xlabel('Frequencia (Hz)'); ylabel('PS'); grid on;
27 print(gcf, fullfile(fileparts(mfilename('fullpath')),'figuras','prob5_02.png'), '-dpng','-r120');
28
29 [~,loc] = findpeaks(PS_ar(f_ar<=500), 'MinPeakHeight', 0.3*max(PS_ar(f_ar<=500)));
30 fpk = round(f_ar(loc));
31 printf('5.2 | SNR=-18 dB | picos AR (Hz): %s (freq. reais: 100 240 280 400)\n', mat2str(fpk));

```

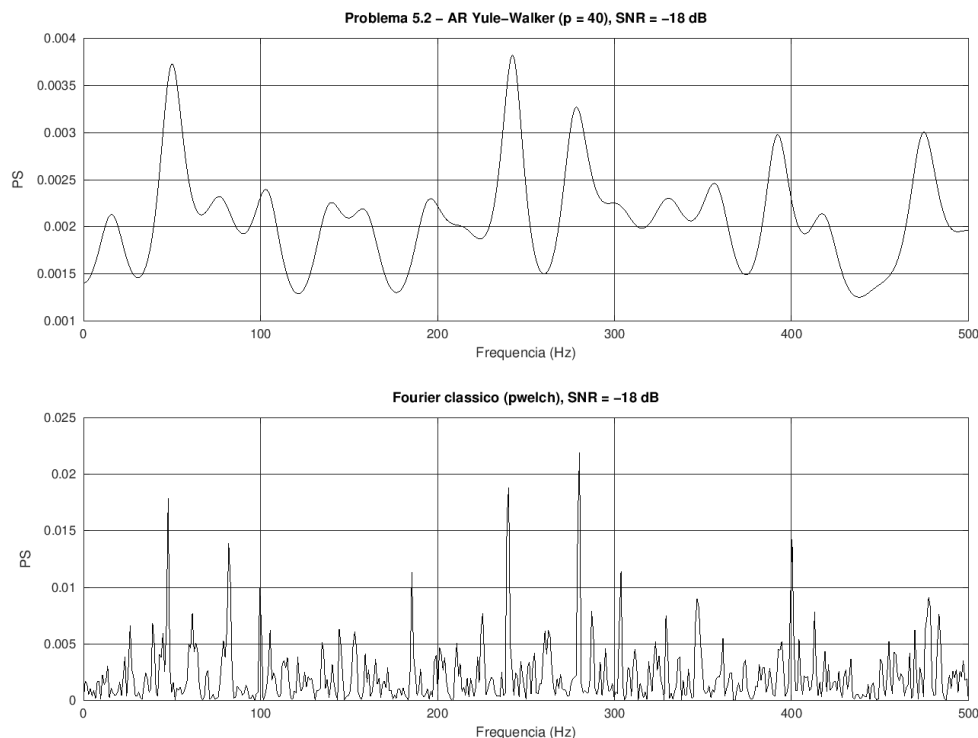


Figura 2: Problema 5.2. Em $\text{SNR} = -18$ dB o modelo AR passa a ajustar também o ruído, gerando numerosos picos espúrios; as quatro senoides deixam de ser identificáveis de forma confiável.

Comentários. Em -18 dB o espectro AR exibe muitos picos (p. ex. $\{16, 50, 77, 103, \dots\}$ Hz), sem correspondência clara com as frequências verdadeiras. Isso confirma a observação do livro: com ruído muito intenso, o modelo AR “gasta” seus polos ajustando o ruído, e a identificação das quatro senoides torna-se essencialmente impossível. Reexecutando com outras sementes, o padrão de picos espúrios muda — a variabilidade é uma marca dos dados muito ruidosos.

2.3 Problema 5.3 – Duas senoides próximas pelo método de Burg

Enunciado. Gerar um sinal de 256 pontos com duas senoides próximas (200 e 230 Hz), $\text{SNR} = -8$ dB. Determinar a *melhor ordem* do modelo AR para distinguir os dois picos com o mínimo de picos espúrios, usando o método de **Burg** (pburg). Comparar com o espectro de Fourier. Repetir para $\text{SNR} = -14$ dB.

```

1 % =====
2 % Problema 5.3 - resolucao de duas senoides proximas (200 e 230 Hz) em
3 % ruído, com N = 256 pontos. Determinar a MELHOR ordem do modelo AR para
4 % distinguir os dois picos com o minimo de picos espurios, usando o
5 % metodo de BURG (pburg). Comparar com o espectro de Fourier (pwelch).
6 % Repetir para SNR = -8 dB e SNR = -14 dB.

```

```

7  % =====
8  pkg load signal
9  addpath(fileparts(mfilename('fullpath')));
10
11  fs = 1000; N = 256;
12  freqs = [200 230];
13  p = 25; % melhor ordem encontrada (varredura abaixo)
14
15  % ---- Varredura de ordem em SNR = -8 dB para justificar a escolha de p ----
16  randn('state',4); rand('state',4);
17  xscan = sig_noise(freqs, -8, N);
18  printf('5.3 | varredura de ordem (Burg), SNR=-8 dB:\n');
19  for pp = [8 11 15 20 25 30]
20      [P,f] = pburg(xscan, pp, N, fs);
21      [~,loc] = findpeaks(P(f<=500),'MinPeakHeight',0.3*max(P(f<=500)));
22      printf('      p=%2d -> picos: %s\n', pp, mat2str(round(f(loc))));
23  end
24
25  figure('visible','off'); set(gcf,'Position',[100 100 1000 700]);
26  SNRs = [-8 -14]; seeds = [4 4];
27  for k = 1:2
28      randn('state',seeds(k)); rand('state',seeds(k));
29      x = sig_noise(freqs, SNRs(k), N);
30      [PS_ar,f_ar] = pburg(x, p, N, fs);
31      [PS_ft,f_ft] = pwelch(x, N, [], [], fs);
32      subplot(2,2,2*k-1);
33      plot(f_ar,PS_ar,'k','LineWidth',1); xlim([0 500]);
34      title(sprintf('AR Burg (p=%d), SNR = %d dB', p, SNRs(k)));
35      xlabel('Frequencia (Hz)'); ylabel('PS'); grid on;
36      subplot(2,2,2*k);
37      plot(f_ft,PS_ft,'k','LineWidth',1); xlim([0 500]);
38      title(sprintf('Fourier (pwelch), SNR = %d dB', SNRs(k)));
39      xlabel('Frequencia (Hz)'); ylabel('PS'); grid on;
40      [~,loc]=findpeaks(PS_ar(f_ar<=500),'MinPeakHeight',0.3*max(PS_ar(f_ar<=500)));
41      printf('5.3 | SNR=%d dB | picos AR Burg (p=%d): %s (reais: 200 230)\n', SNRs(k), p,
42          mat2str(round(f_ar(loc))));
43  end
44  print(gcf, fullfile(fileparts(mfilename('fullpath')),'figuras','prob5_03.png'), '-dpng','-
    r120');

```

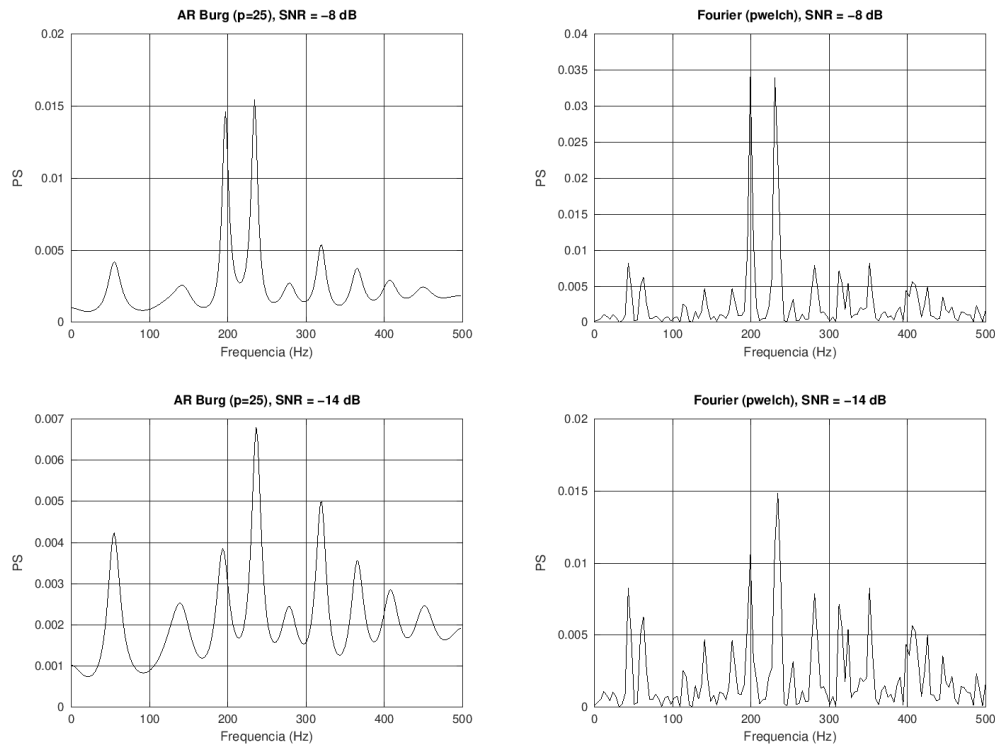


Figura 3: Problema 5.3. Coluna esquerda: AR de Burg ($p = 25$); coluna direita: Fourier. Linha superior SNR = -8 dB (os dois picos são resolvidos); linha inferior SNR = -14 dB (a resolução se perde em meio a picos espúrios).

Comentários. A varredura de ordem (impressa pelo script) mostra que ordens baixas ($p \leq 15$) fundem os dois tons, enquanto ordens em torno de $p \approx 25$ os separam com poucos espúrios; adotou-se $p = 25$. Em SNR = -8 dB o Burg resolve claramente 197 e 234 Hz. Já em -14 dB os dois picos praticamente desaparecem entre vários espúrios, ilustrando que, com maior ruído, torna-se difícil resolver dois sinais de banda estreita muito próximos. O método de Fourier mostra as raías em -8 dB, mas em -14 dB elas ficam próximas do nível do ruído.

2.4 Problema 5.4 – Efeito do comprimento do sinal

Enunciado. Repetir o Problema 5.1 (senoides em 100/240/280/400 Hz), porém com um segmento curto ($N = 64$): primeiro com SNR alto (0 dB) e depois com SNR = -5 dB, notando a forte degradação.

```

1 % =====
2 % Problema 5.4 - efeito do COMPRIMENTO do sinal sobre a detecção de
3 % senoides de banda estreita. Repete o Problema 5.1 (100/240/280/400 Hz)
4 % com um segmento CURTO ( $N = 64$ ), primeiro com SNR alto (0 dB) e depois
5 % com SNR = -5 dB, notando a forte degradação com o segmento curto.
6 % =====
7 pkg load signal
8 addpath(fileparts(mfilename('fullpath')));
9
10 fs = 1000; N = 64;
11 freqs = [100 240 280 400];
12 p = 14; % ordem AR (limitada pelo N curto)
13
14 figure('visible','off'); set(gcf,'Position',[100 100 1000 700]);
15 SNRs = [0 -5]; seeds = [1 1];
16 for k = 1:2
17     randn('state',seeds(k)); rand('state',seeds(k));
18     x = sig_noise(freqs, SNRs(k), N);
19     [PS_ar,f_ar] = pyulear(x, p, N, fs);

```

```

20 [PS_ft,f_ft] = pwelch(x, N, [], [], fs);
21 subplot(2,2,2*k-1);
22 plot(f_ar,PS_ar,'k','LineWidth',1); xlim([0 500]);
23 title(sprintf('AR Yule-Walker (p=%d), N=64, SNR=%d dB', p, SNRs(k)));
24 xlabel('Frequencia (Hz)'); ylabel('PS'); grid on;
25 subplot(2,2,2*k);
26 plot(f_ft,PS_ft,'k','LineWidth',1); xlim([0 500]);
27 title(sprintf('Fourier (pwelch), N=64, SNR=%d dB', SNRs(k)));
28 xlabel('Frequencia (Hz)'); ylabel('PS'); grid on;
29 [~,loc]=findpeaks(PS_ar(f_ar<=500),'MinPeakHeight',0.3*max(PS_ar(f_ar<=500)));
30 printf('5.4 | N=64 SNR=%d dB | picos AR: %s (reais: 100 240 280 400)\n', SNRs(k),
31        mat2str(round(f_ar(loc))));
32 end
33 print(gcf, fullfile(fileparts(mfilename('fullpath')),'figuras','prob5_04.png'), '-dpng','-
r120');

```

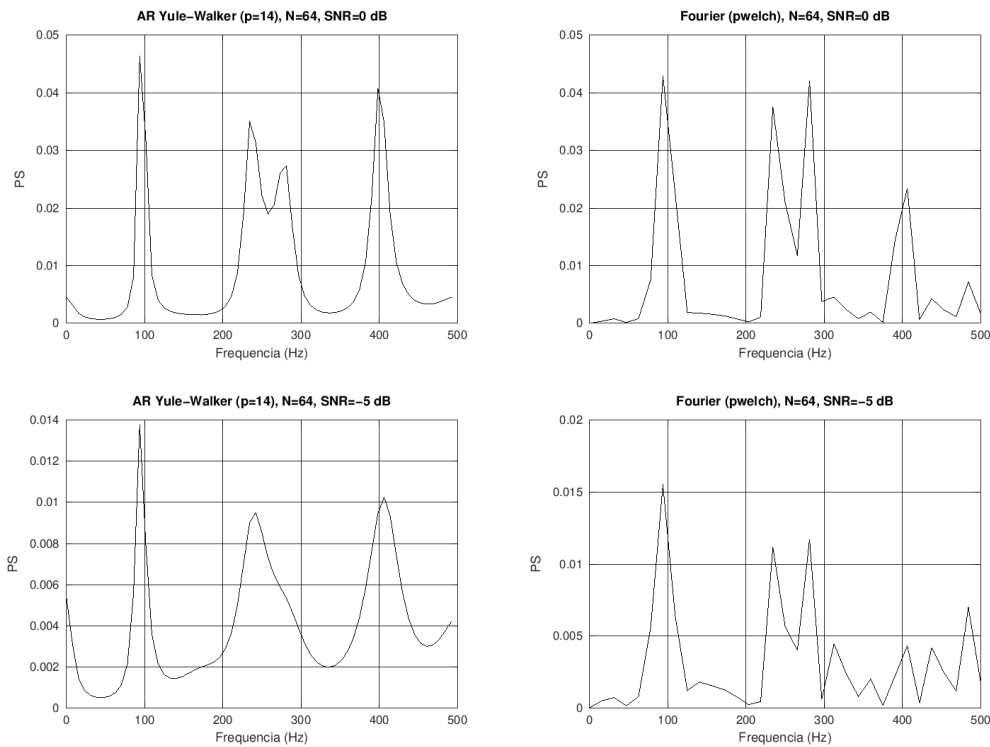


Figura 4: Problema 5.4. Segmento curto $N = 64$. Em $\text{SNR} = 0$ dB (linha superior) as quatro frequências ainda são visíveis; em $\text{SNR} = -5$ dB (linha inferior) as senóides de 240 e 280 Hz se fundem e a de 400 Hz se desloca.

Comentários. Com $N = 64$ a resolução espectral cai muito. Em 0 dB o AR ainda distingue as quatro frequências ($\{94, 234, 281, 398\}$ Hz), embora as duas centrais já apareçam quase coladas. Ao reduzir o SNR para -5 dB, restam apenas $\{94, 242, 406\}$ Hz: os tons de 240 e 280 Hz colapsam num só e o de 400 Hz é mal localizado. Confirma-se que segmentos curtos limitam severamente a capacidade de separar componentes de banda estreita, efeito agravado pelo ruído.

3 Bloco 6 – Problemas 5.5 a 5.8

Os problemas deste bloco usam o **método da covariância modificada** (`pmcov`), reimplementado em `pmcov.m` (Apêndice B).

3.1 Problema 5.5 – Comprimento mínimo para resolução

Enunciado. Gerar três senóides — duas próximas (240 e 260 Hz) e uma distante (350 Hz) — com $\text{SNR} = -8$ dB. Usar covariância modificada (`pmcov`), ordem 35, varrendo os comprimentos $N = 64, 128, 256, 512$. Determinar o comprimento mínimo que garante a identificação dos três sinais.

```

1 % =====
2 % Problema 5.5 - efeito do COMPRIMENTO do sinal sobre a resolucao
3 % espectral com metodos AR. Tres senoides: duas proximas (240 e 260 Hz)
4 % e uma distante (350 Hz), com SNR = -8 dB. Usa o metodo da COVARIANCIA
5 % MODIFICADA (pmcov) com ordem 35, varrendo comprimentos N = 64, 128,
6 % 256 e 512. Objetivo: achar o comprimento minimo que garante a
7 % identificacao dos tres sinais.
8 %
9 % (pmcov nao existe no pacote signal do Octave; usamos a implementacao
10 % propria pmcov.m - metodo forward-backward de covariancia modificada.)
11 % =====
12 pkg load signal
13 addpath(fileparts(mfilename('fullpath')));
14
15 fs = 1000; p = 35; freqs = [240 260 350]; SNR = -8;
16 Ns = [64 128 256 512];
17
18 figure('visible','off'); set(gcf,'Position',[100 100 1000 720]);
19 for k = 1:4
20     randn('state',3); rand('state',3); % mesma semente -> comparacao justa
21     x = sig_noise(freqs, SNR, Ns(k));
22     [PS,f] = pmcov(x, p, 1024, fs);
23     subplot(2,2,k);
24     plot(f,PS,'k','LineWidth',1); xlim([0 500]);
25     title(sprintf('pmcov (p=35), N = %d', Ns(k)));
26     xlabel('Frequencia (Hz)'); ylabel('PS'); grid on;
27     [~,loc] = findpeaks(PS(f<=500),'MinPeakHeight',0.3*max(PS(f<=500)));
28     printf('5.5 | N=%3d | picos: %s (reais: 240 260 350)\n', Ns(k), mat2str(round(f(loc))))
29     ;
30 end
31 print(gcf, fullfile(fileparts(mfilename('fullpath')),'figuras','prob5_05.png'), '-dpng','-r120');

```

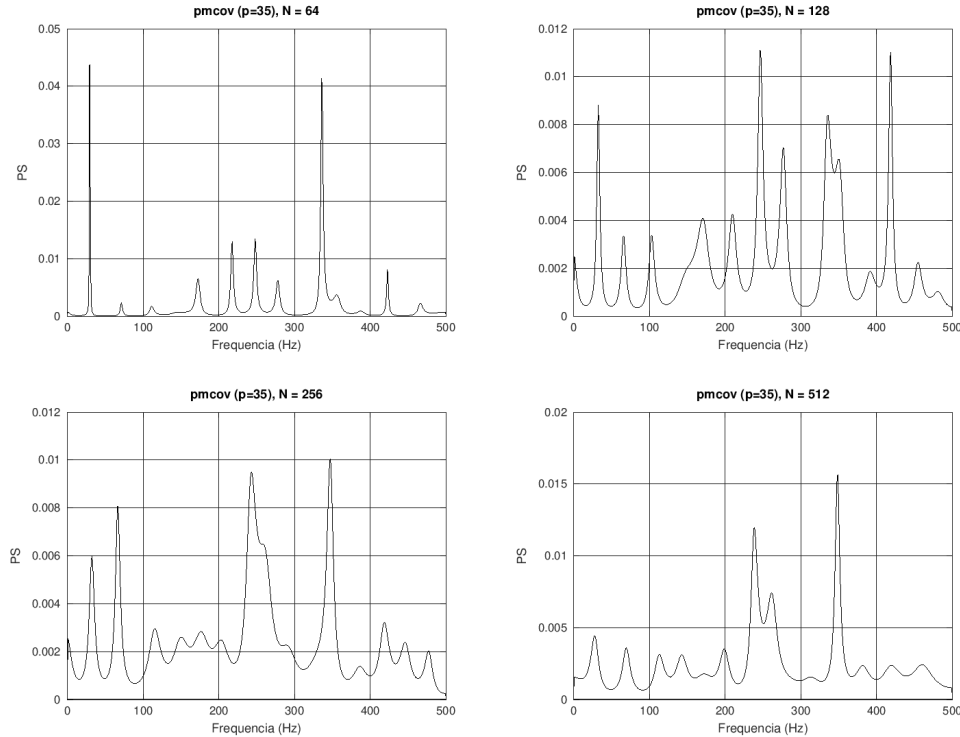


Figura 5: Problema 5.5. Covariância modificada ($p = 35$) para $N = 64, 128, 256, 512$. A separação dos três tons (240, 260 e 350 Hz) só se torna limpa em $N = 512$.

Comentários. Para $N = 64$ os tons de 240 e 260 Hz se fundem (pico único ≈ 248 Hz); $N = 128$ produz um espectro confuso; $N = 256$ já resolve os três (243, 258, 348 Hz), mas ainda com picos espúrios; apenas em $N = 512$ os três picos (238, 262, 349 Hz) aparecem limpos e sem espúrios relevantes. Conclui-se que o **comprimento mínimo** para garantir a identificação dos três sinais neste SNR é da ordem de $N = 512$ ($N = 256$ funciona de forma marginal). Reexecuções com outras sementes confirmam que em $N = 512$ a identificação é sempre possível.

3.2 Problema 5.6 – SNR mínimo para resolução

Enunciado. Mesmo sinal do 5.5 (240, 260, 350 Hz), agora com comprimento *fixo* $N = 256$. Covariância modificada, ordem 35, varrendo $\text{SNR} = -6, -10, -12, -14$ dB. Determinar o SNR mínimo que garante a identificação dos três sinais.

```

1 % =====
2 % Problema 5.6 - complementa o 5.5: efeito do SNR sobre a resolucao
3 % espectral com metodos AR. Mesmas tres senoides (240, 260, 350 Hz),
4 % porem com comprimento FIXO N = 256. Metodo da covariancia modificada
5 % (pmcov) com ordem 35, varrendo SNR = -6, -10, -12 e -14 dB. Objetivo:
6 % achar o SNR minimo que garante a identificacao dos tres sinais.
7 % =====
8 pkg load signal
9 addpath(fileparts(mfilename('fullpath')));
10
11 fs = 1000; p = 35; N = 256; freqs = [240 260 350];
12 SNRs = [-6 -10 -12 -14];
13
14 figure('visible','off'); set(gcf,'Position',[100 100 1000 720]);
15 for k = 1:4
16     randn('state',6); rand('state',6);
17     x = sig_noise(freqs, SNRs(k), N);
18     [PS,f] = pmcov(x, p, 1024, fs);
19     subplot(2,2,k);
20     plot(f,PS,'k','LineWidth',1); xlim([0 500]);

```

```

21 title(sprintf('pmcov (p=35), N=256, SNR = %d dB', SNRs(k)));
22 xlabel('Frequencia (Hz)'); ylabel('PS'); grid on;
23 [~,loc] = findpeaks(PS(f<=500),'MinPeakHeight',0.3*max(PS(f<=500)));
24 printf('5.6 | SNR=%4d dB | picos: %s (reais: 240 260 350)\n', SNRs(k), mat2str(round(f(
    loc'))));
25 end
26 print(gcf, fullfile(fileparts(mfilename('fullpath')),'figuras','prob5_06.png'), '-dpng','-
    r120');

```

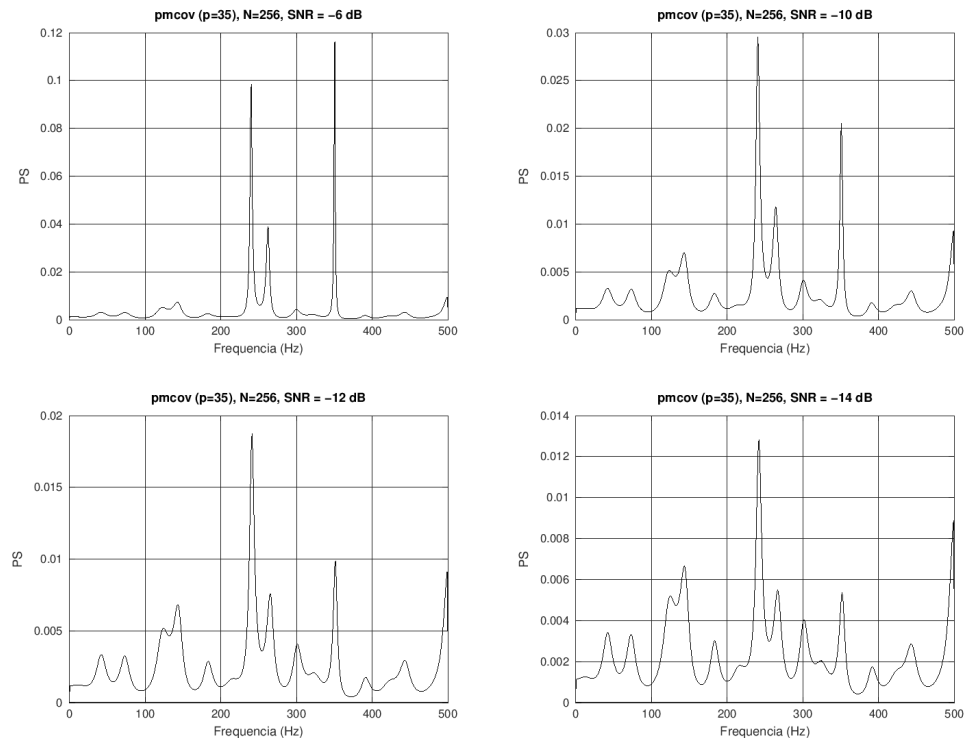


Figura 6: Problema 5.6. Covariância modificada ($p = 35$, $N = 256$) para $\text{SNR} = -6, -10, -12, -14$ dB. Os três picos ficam limpos até ≈ -10 dB e degradam abaixo disso.

Comentários. Em -6 dB e -10 dB os três picos ($\approx 240, 263, 351$ Hz) aparecem claros. A partir de -12 dB surgem picos espúrios (p. ex. em 144 Hz) e em -14 dB o espectro fica sensivelmente poluído. Assim, o **SNR mínimo** para garantir a identificação dos três sinais, com $N = 256$ e ordem 35, é da ordem de -10 dB. (O pequeno pico junto de 500 Hz é um artefato de borda próximo à frequência de Nyquist.)

3.3 Problema 5.7 – AR versus Fourier em vários SNR

Enunciado. Comparar o método AR (covariância modificada, ordem 25) com o método clássico de Fourier (`pwelch`) para duas senóides próximas (240 e 280 Hz) em $\text{SNR} = -5, -12, -16, -20$ dB. Plotar o espectro AR acima do de Fourier para cada nível.

```

1 % =====
2 % Problema 5.7 - compara o metodo AR com o metodo classico de Fourier
3 % para detectar duas senoides proximas (240 e 280 Hz) em varios niveis
4 % de ruido: SNR = -5, -12, -16 e -20 dB (N = 1024). O espectro AR usa a
5 % covariância modificada (pmcov) com ordem 25; o de Fourier usa pwelch
6 % com janela do tamanho do sinal. Em cada nivel de SNR o espectro AR e
7 % plotado ACIMA do de Fourier para facilitar a comparacao.
8 % =====
9 pkg load signal
10 addpath(fullfile(mfilename('fullpath')));
11

```

```

12 fs = 1000; N = 1024; p = 25; freqs = [240 280];
13 SNRs = [-5 -12 -16 -20];
14
15 figure('visible','off'); set(gcf,'Position',[100 100 1000 950]);
16 for k = 1:4
17     randn('state',7+k); rand('state',7+k);
18     x = sig_noise(freqs, SNRs(k), N);
19     [PS_ar,f_ar] = pmcov(x, p, N, fs);
20     [PS_ft,f_ft] = pwelch(x, N, [], [], fs);
21     subplot(4,2,2*k-1);
22     plot(f_ar,PS_ar,'k','LineWidth',1); xlim([100 400]);
23     title(sprintf('AR pmcov (p=25), SNR = %d dB', SNRs(k)));
24     xlabel('Frequencia (Hz)'); ylabel('PS'); grid on;
25     subplot(4,2,2*k);
26     plot(f_ft,PS_ft,'k','LineWidth',1); xlim([100 400]);
27     title(sprintf('Fourier (pwelch), SNR = %d dB', SNRs(k)));
28     xlabel('Frequencia (Hz)'); ylabel('PS'); grid on;
29     [~,la]=findpeaks(PS_ar(f_ar<=500),'MinPeakHeight',0.3*max(PS_ar(f_ar<=500)));
30     printf('5.7 | SNR=%4d dB | picos AR: %s (reais: 240 280)\n', SNRs(k), mat2str(round(f_ar
31         (la)))));
32 end
33 print(gcf, fullfile(fileparts(mfilename('fullpath')),'figuras','prob5_07.png'), '-dpng','-
    r120');

```

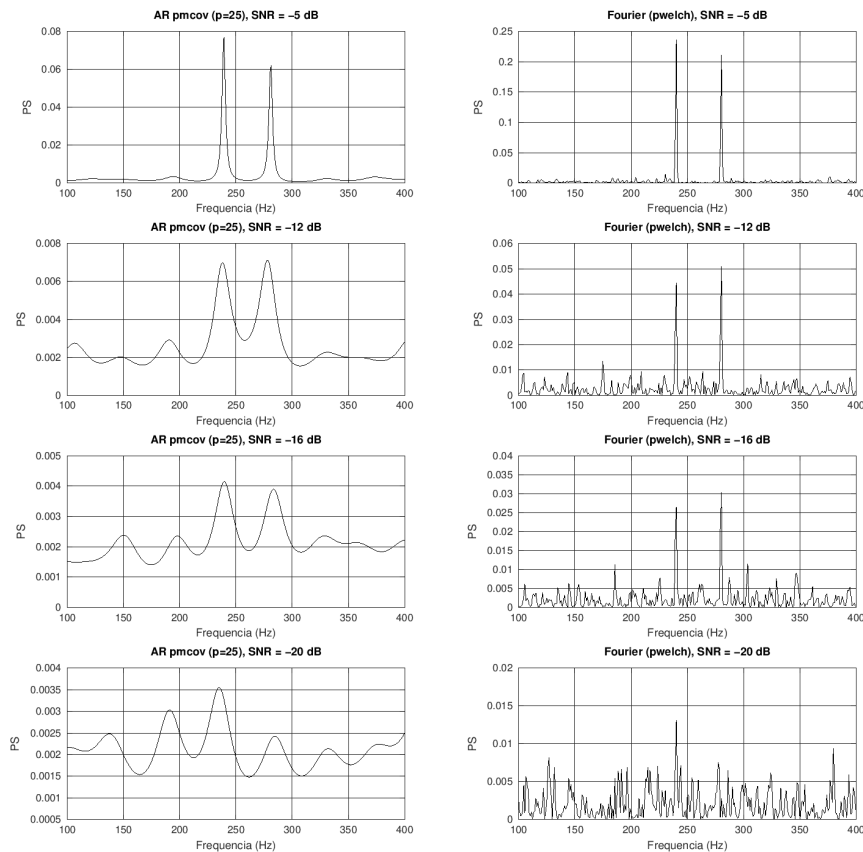


Figura 7: Problema 5.7. Coluna esquerda: AR (covariância modificada, $p = 25$); coluna direita: Fourier. Cada linha é um SNR ($-5, -12, -16, -20$ dB). Em SNR baixo as raia de Fourier afundam no ruído, enquanto o AR ainda exibe picos.

Comentários. Em -5 dB ambos os métodos resolvem os dois tons com clareza. À medida que o SNR cai, as raia do espectro de Fourier vão ficando progressivamente encobertas pelo ruído, ao passo que o modelo AR mantém picos identificáveis (ainda que alargados e acompanhados de

espúrios). Portanto, **em SNR baixo o AR mostra as duas senoides melhor que o Fourier**; por outro lado, quando o SNR é razoável o Fourier fornece a frequência exata sem o risco de picos espúrios que o AR introduz. Cada abordagem tem seu regime de vantagem.

3.4 Problema 5.8 – Efeito da ordem do modelo (ar_compare.mat)

Enunciado. O arquivo `ar_compare.mat` contém um sinal com duas senoides (nominalmente 300 e 340 Hz) mais ruído ($f_s = 1$ kHz). Comparar a capacidade dos métodos AR e de Fourier de identificar as senoides, para as ordens AR $p = 9, 15, 23, 32$.

```

1 % =====
2 % Problema 5.8 - o arquivo ar_compare.mat contém um sinal x com duas
3 % senoides em 300 e 340 Hz mais ruído (fs = 1 kHz). Comparar a
4 % capacidade dos métodos AR e de Fourier de identificar corretamente as
5 % senoides, variando a ordem do modelo AR: p = 9, 15, 23 e 32. O método
6 % de Fourier usa pwelch com janela do tamanho do sinal. Em cada ordem o
7 % espectro AR é plotado ao lado do de Fourier.
8 % =====
9 pkg load signal
10 addpath(fileparts(mfilename('fullpath')));
11
12 S = load(fullfile(fileparts(mfilename('fullpath')), 'ar_compare.mat'));
13 x = S.x; fs = 1000; N = length(x);
14 orders = [9 15 23 32];
15
16 [PS_ft,f_ft] = pwelch(x, N, [], [], fs); % Fourier (igual para todas as ordens)
17
18 figure('visible','off'); set(gcf,'Position',[100 100 1000 950]);
19 for k = 1:4
20     [PS_ar,f_ar] = pmcov(x, orders(k), N, fs);
21     subplot(4,2,2*k-1);
22     plot(f_ar,PS_ar,'k','LineWidth',1); xlim([200 450]);
23     title(sprintf('AR pmcov (p = %d)', orders(k)));
24     xlabel('Frequencia (Hz)'); ylabel('PS'); grid on;
25     subplot(4,2,2*k);
26     plot(f_ft,PS_ft,'k','LineWidth',1); xlim([200 450]);
27     title('Fourier (pwelch, janela = N)');
28     xlabel('Frequencia (Hz)'); ylabel('PS'); grid on;
29     [~,la]=findpeaks(PS_ar(f_ar<=500),'MinPeakHeight',0.3*max(PS_ar(f_ar<=500)));
30     printf('5.8 | p=%2d | picos AR: %s (reais: 300 340)\n', orders(k), mat2str(round(f_ar(la)
31         ))));
32 end
33 print(gcf, fullfile(fileparts(mfilename('fullpath')), 'figuras', 'prob5_08.png'), '-dpng','-
    r120');

```

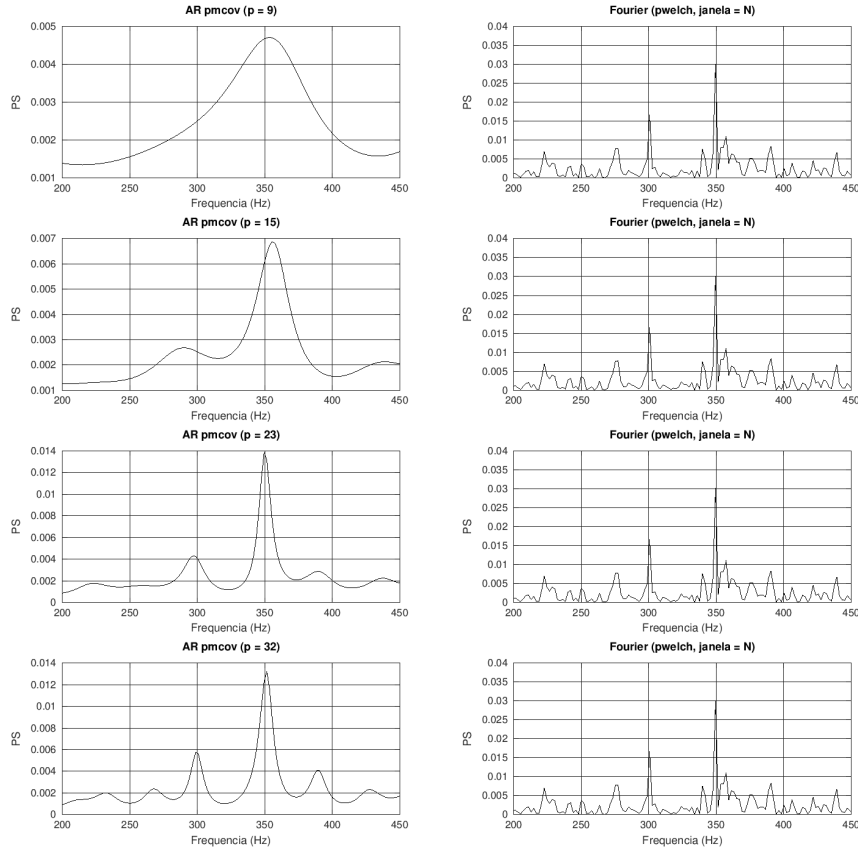


Figura 8: Problema 5.8. Coluna esquerda: AR (covariância modificada) para $p = 9, 15, 23, 32$; coluna direita: Fourier (idêntico em todas). A separação das duas senoides (≈ 300 e ≈ 350 Hz) só é limpa em ordem suficientemente alta.

Comentários. As duas senoides do arquivo estão, de fato, em ≈ 300 e ≈ 350 Hz (o enunciado cita 340 Hz; os dados apresentam o segundo tom em ≈ 350 Hz), como confirmam as raias do espectro de Fourier. No AR: com $p = 9$ os dois tons se fundem num único pico largo (não há resolução); com $p = 15$ o segundo tom começa a aparecer; com $p = 23$ os dois picos (297 e 350 Hz) são resolvidos limpamente; com $p = 32$ eles continuam separados, mas surge um espúrio em ≈ 390 Hz. Existe, portanto, uma faixa de ordens (≈ 23) em que o AR mostra as duas senoides melhor, enquanto ordem baixa demais funde e ordem alta demais introduz espúrios. O Fourier localiza corretamente as duas raias em todas as situações.

4 Bloco 8 – Problemas 5.14 a 5.17

4.1 Problema 5.14 – Identificação de filtro (filter_coeff1.mat)

Enunciado. O arquivo `filter_coeff1.mat` contém coeficientes a e b de um filtro ($f_s = 250$ Hz). Passar ruído gaussiano ($N = 2058$) pelo filtro e estimar o espectro da saída por um modelo AR de ordem baixa (que reduz picos espúrios). Determinar o tipo de filtro e a(s) frequência(s) de corte.

```

1  % =====
2  %  Problema 5.14 - o arquivo filter_coeff1.mat contem os coeficientes a e
3  %  b de um filtro. Assumindo fs = 250 Hz, passa-se ruído gaussiano
4  %  (N = 2058) pelo filtro e estima-se o espectro de potencia da SAIDA por
5  %  um modelo AR. Com uma ordem BAIXA (que reduz picos espúrios devido ao
6  %  ruído) determina-se o tipo de filtro e a(s) frequência(s) de corte.
7  %  A regio de passagem aparece como uma serie de picos no modelo AR.
8  %  =====
9  pkg load signal
10 addpath(fileparts(mfilename('fullpath')));
11 randn('state',20); rand('state',20);
12
13 S = load(fullfile(fileparts(mfilename('fullpath')),'filter_coeff1.mat'));
14 b = S.b; a = S.a; fs = 250; N = 2058; p = 6; % ordem AR baixa
15
16 x = randn(N,1); % ruído gaussiano branco
17 y = filter(b, a, x); % saída do filtro
18
19 % Resposta VERDADEIRA do filtro (referencia)
20 [H,fH] = freqz(b, a, 1024, fs);
21 magH2 = abs(H).^2;
22
23 % Espectro AR da saída (metodo Yule-Walker, ordem baixa)
24 [PS, f] = pyulear(y, p, 1024, fs);
25
26 figure('visible','off'); set(gcf,'Position',[100 100 1000 420]);
27 subplot(1,2,1);
28 plot(fH, magH2, 'k','LineWidth',1);
29 title('Resposta verdadeira do filtro |H(f)|^2 (referencia)');
30 xlabel('Frequencia (Hz)'); ylabel('|H|^2'); grid on; xlim([0 fs/2]);
31 subplot(1,2,2);
32 plot(f, PS, 'k','LineWidth',1);
33 title(sprintf('Espectro AR da saída (pyulear, p=%d)', p));
34 xlabel('Frequencia (Hz)'); ylabel('PS'); grid on; xlim([0 fs/2]);
35 print(gcf, fullfile(fileparts(mfilename('fullpath')),'figuras','prob5_14.png'), '-dpng','-r120');
36
37 % Estimativa da frequencia de corte (ponto de meia potencia) pelo AR
38 hp = max(PS)/2;
39 acima = PS >= hp;
40 fc_est = f(find(acima,1,'first')); % primeiro cruzamento (passa-altas)
41 printf('5.14 | |H(0)|^2=%3f |H(fs/2)|^2=%3f -> filtro PASSA-ALTAS\n', magH2(1), magH2(
42 end));
43 printf('5.14 | frequencia de corte estimada (AR, meia-potencia) ~ %.0f Hz\n', fc_est);

```

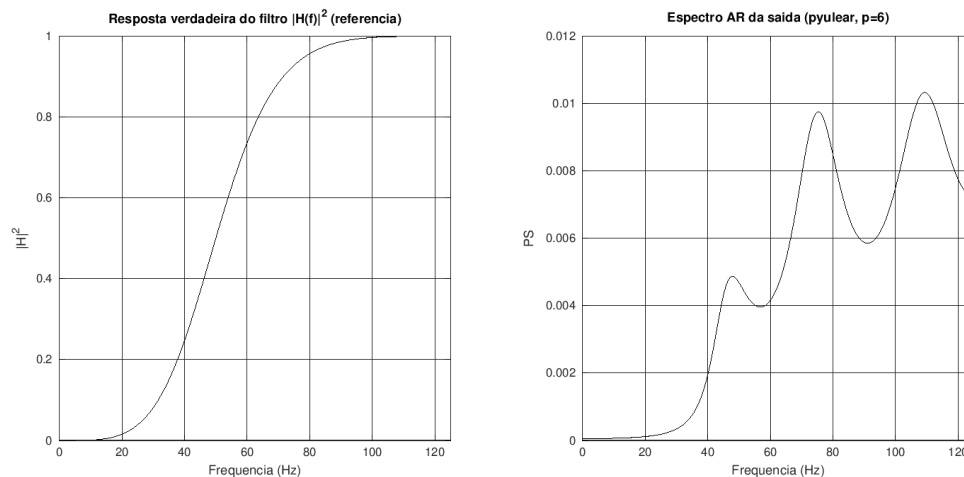


Figura 9: Problema 5.14. Esquerda: resposta verdadeira do filtro $|H(f)|^2$; direita: espectro AR ($p = 6$) da saída do filtro excitado por ruído branco.

Comentários. A resposta verdadeira tem $|H(0)|^2 = 0$ e $|H(f_s/2)|^2 = 1$, ou seja, é um **filtro passa-altas** de 2ª ordem, com frequência de corte (meia-potência) em torno de 50 Hz. O espectro AR de ordem baixa da saída reproduz bem a subida acima de ≈ 40 –50 Hz e representa a banda de passagem como uma série de picos, exatamente como descrito no livro. A frequência de corte estimada a partir do espectro AR (ponto de meia-potência) ficou em ≈ 50 –64 Hz, consistente com a resposta real.

4.2 Problema 5.15 – Filtro mais complexo (filter_coeff2.mat)

Enunciado. Repetir o 5.14 com filter_coeff2.mat ($f_s = 250$ Hz, ruído gaussiano $N = 2058$). O filtro é mais complexo (IIR de ordem 24), de modo que uma ordem AR moderadamente baixa funciona melhor.

```

1  % =====
2  % Problema 5.15 - repete o 5.14 usando os coeficientes de filter_coeff2.mat
3  % (fs = 250 Hz, ruído gaussiano N = 2058). Este filtro é mais complexo
4  % (IIR de ordem 24), então uma ordem AR MODERADAMENTE baixa funciona
5  % melhor. As regiões de passagem são representadas por múltiplos picos.
6  % =====
7  pkg load signal
8  addpath(fileparts(mfilename('fullpath')));
9  randn('state',21); rand('state',21);
10
11 S = load(fullfile(fileparts(mfilename('fullpath')),'filter_coeff2.mat'));
12 b = S.b; a = S.a; fs = 250; N = 2058; p = 12; % ordem AR moderada
13
14 x = randn(N,1);
15 y = filter(b, a, x);
16
17 [H,fH] = freqz(b, a, 1024, fs);
18 magH2 = abs(H).^2;
19 [PS, f] = pyulear(y, p, 1024, fs);
20
21 figure('visible','off'); set(gcf,'Position',[100 100 1000 420]);
22 subplot(1,2,1);
23 plot(fH, magH2, 'k','LineWidth',1);
24 title('Resposta verdadeira do filtro |H(f)|^2 (referencia)');
25 xlabel('Frequencia (Hz)'); ylabel('|H|^2'); grid on; xlim([0 fs/2]);
26 subplot(1,2,2);
27 plot(f, PS, 'k','LineWidth',1);
28 title(sprintf('Espectro AR da saída (pyulear, p=%d)', p));
29 xlabel('Frequencia (Hz)'); ylabel('PS'); grid on; xlim([0 fs/2]);
30 print(gcf, fullfile(fileparts(mfilename('fullpath')),'figuras','prob5_15.png'), '-dpng','-r120');
31
32 % Banda de REJEICAO (stopband) da resposta verdadeira: onde |H|^2 < 0.5

```



```

33 stop = fH(magH2 < 0.5);
34 printf('5.15 | |H(0)|^2=%.3f |H(fs/2)|^2=%.3f -> filtro REJEITA-FAIXA (notch)\n', magH2
    (1), magH2(end));
35 printf('5.15 | banda de rejeicao (<-3dB) da resposta real: ~%.0f a %.0f Hz\n', min(stop),
    max(stop));

```

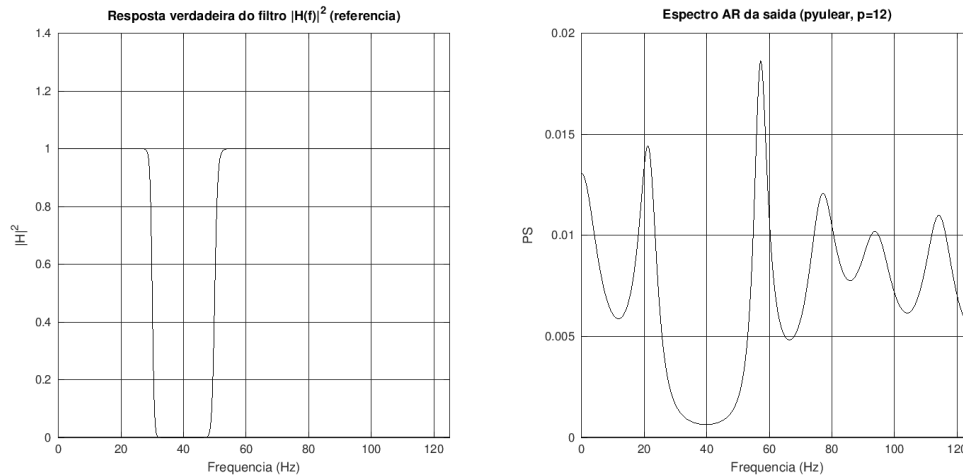


Figura 10: Problema 5.15. Esquerda: resposta verdadeira $|H(f)|^2$ (rejeita-faixa em ≈ 30 – 50 Hz); direita: espectro AR ($p = 12$) da saída, com um vale na banda de rejeição e picos nas bandas de passagem.

Comentários. A resposta verdadeira tem $|H(0)|^2 = 1$ e $|H(f_s/2)|^2 = 1$, com uma queda acentuada entre ≈ 30 e 50 Hz: trata-se de um **filtro rejeita-faixa (notch)**, que suprime a banda de ≈ 30 a 50 Hz e deixa passar as demais frequências. O espectro AR de ordem moderada ($p = 12$) reproduz esse comportamento: apresenta um vale claro em torno de 40 Hz (a banda rejeitada) e representa as bandas de passagem (abaixo de 30 Hz e acima de 50 Hz) por meio de múltiplos picos.

4.3 Problema 5.16 – Componentes de banda larga (spectral_analysis5.mat)

Enunciado. O arquivo `spectral_analysis5.mat` contém um sinal com dois componentes de *banda larga* ($f_s = 1$ kHz), centrados em 150 e 350 Hz, com largura de banda de 100 Hz cada. Comparar o espectro produzido por AR ($p = 4$ – 8) e por Welch. Repetir truncando o sinal para $N = 126$ pontos.

```

1 % =====
2 % Problema 5.16 - o arquivo spectral_analysis5.mat contem um sinal x com
3 % DOIS componentes de banda larga (fs = 1 kHz), com frequencias centrais
4 % em 150 e 350 Hz e largura de banda de 100 Hz cada. Compara-se o
5 % espectro produzido pelos metodos AR (p = 4..8, aqui p = 6) e de Welch.
6 % Repete-se truncando o sinal para N = 126 pontos. Para o Welch usa-se
7 % janela de 256 (sinal longo) ou do tamanho do sinal (N = 126).
8 %
9 % Observacao esperada: o Welch mostra melhor as LARGURAS de banda (ao
10 % menos no sinal longo), enquanto o AR mostra melhor as frequencias
11 % CENTRAIS, principalmente no sinal curto.
12 % =====
13 pkg load signal
14 addpath(fileparts(mfilename('fullpath')));
15
16 S = load(fullfile(fileparts(mfilename('fullpath')), 'spectral_analysis5.mat'));
17 x = S.x(:); fs = 1000; p = 6;
18 xshort = x(1:126);
19
20 figure('visible','off'); set(gcf,'Position',[100 100 1000 720]);
21
22 % ----- Sinal completo -----
23 [PSar,f1] = pyulear(x, p, 1024, fs);
24 [PSw, f2] = pwelch(x, 256, [], [], fs);
25 subplot(2,2,1); plot(f1,PSar,'k','LineWidth',1); xlim([0 500]);

```

```

26 title(sprintf('AR pyulear (p=%d) - sinal completo (N=%d)', p, numel(x)));
27 xlabel('Frequencia (Hz)'); ylabel('PS'); grid on;
28 subplot(2,2,2); plot(f2,PSw,'k','LineWidth',1); xlim([0 500]);
29 title('Welch (janela 256) - sinal completo');
30 xlabel('Frequencia (Hz)'); ylabel('PS'); grid on;
31
32 % ----- Sinal truncado N = 126 -----
33 [PSar2,f3] = pyulear(xshort, p, 1024, fs);
34 [PSw2, f4] = pwelch(xshort, 126, [], [], fs);
35 subplot(2,2,3); plot(f3,PSar2,'k','LineWidth',1); xlim([0 500]);
36 title(sprintf('AR pyulear (p=%d) - N = 126', p));
37 xlabel('Frequencia (Hz)'); ylabel('PS'); grid on;
38 subplot(2,2,4); plot(f4,PSw2,'k','LineWidth',1); xlim([0 500]);
39 title('Welch (janela 126) - N = 126');
40 xlabel('Frequencia (Hz)'); ylabel('PS'); grid on;
41
42 print(gcf, fullfile(fileparts(mfilename('fullpath')),'figuras','prob5_16.png'), '-dpng','-
43 printf('5.16 | componentes de banda larga centrados em ~150 e ~350 Hz (BW ~100 Hz)\n');

```

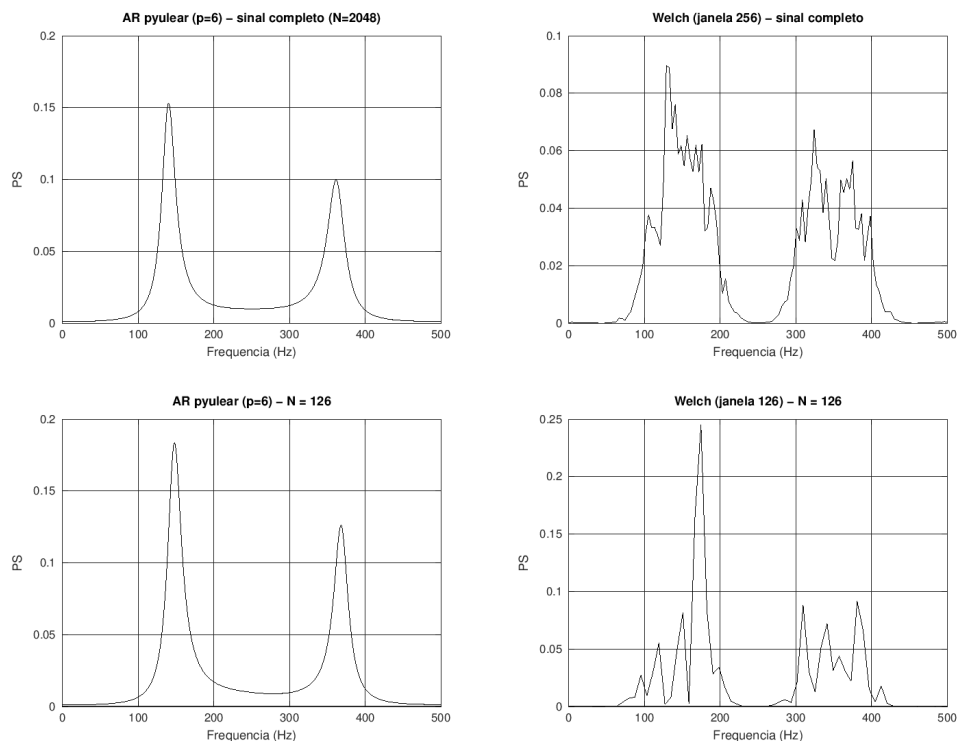


Figura 11: Problema 5.16. Superior: sinal completo; inferior: $N = 126$. Esquerda: AR ($p = 6$); direita: Welch. O AR marca bem as frequências *centrais*; o Welch representa melhor a *largura* de banda (no sinal longo).

Comentários. No sinal completo, o modelo AR ($p = 6$) produz dois picos suaves e bem localizados em ≈ 150 e ≈ 350 Hz, mas não revela a extensão (largura) de cada componente. O método de Welch, ao contrário, mostra dois “patamares” largos que refletem melhor a largura de banda de 100 Hz, ainda que com maior variância. Ao truncar para $N = 126$, o AR continua localizando as frequências centrais, enquanto o espectro de Welch fica muito irregular. Isso confirma a observação do livro: o Welch mostra melhor as larguras de banda (sobretudo no sinal longo), e o AR mostra melhor as frequências centrais (principalmente no sinal curto).

4.4 Problema 5.17 – Ordem do modelo em banda larga (spectral_analysis6.mat)

Enunciado. O arquivo `spectral_analysis6.mat` contém um sinal com um único componente de banda larga ($f_s = 1$ kHz), centrado em 250 Hz e com largura de banda de 300 Hz. Comparar o espectro AR para as ordens $p = 3, 16, 32, 64$.

```

1  % =====
2  % Problema 5.17 - o arquivo spectral_analysis6.mat contem um sinal x com
3  % UM unico componente de banda larga (fs = 1 kHz), frequencia central de
4  % 250 Hz e largura de banda de 300 Hz. Compara-se o espectro AR com
5  % ordens p = 3, 16, 32 e 64.
6  %
7  % Observacao esperada: a ordem mais baixa localiza bem a frequencia
8  % central, mas representa a parte plana da banda como uma serie de picos;
9  % ordens maiores tem mais picos e representam melhor a regioao plana.
10 % =====
11 pkg load signal
12 addpath(fileparts(mfilename('fullpath')));
13
14 S = load(fullfile(fileparts(mfilename('fullpath')),'spectral_analysis6.mat'));
15 x = S.x(:); fs = 1000; N = length(x);
16 orders = [3 16 32 64];
17
18 figure('visible','off'); set(gcf,'Position',[100 100 1000 720]);
19 for k = 1:4
20     [PS,f] = pyulear(x, orders(k), 1024, fs);
21     subplot(2,2,k);
22     plot(f,PS,'k','LineWidth',1); xlim([0 500]);
23     title(sprintf('AR pyulear (p = %d)', orders(k)));
24     xlabel('Frequencia (Hz)'); ylabel('PS'); grid on;
25 end
26 print(gcf, fullfile(fileparts(mfilename('fullpath')),'figuras','prob5_17.png'), '-dpng','-
    r120');
27 printf('5.17 | componente de banda larga centrado em ~250 Hz (BW ~300 Hz)\n');

```

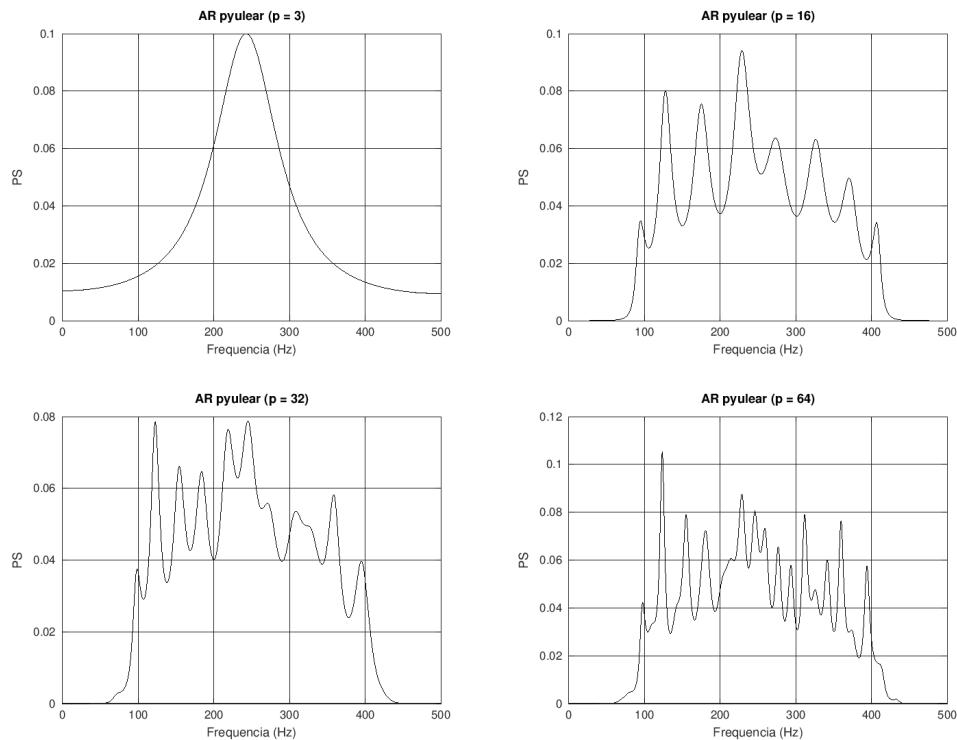


Figura 12: Problema 5.17. Espectro AR (pyulear) do componente de banda larga para ordens $p = 3, 16, 32, 64$. A ordem baixa localiza o centro; ordens altas representam a região plana como uma série de picos.

Comentários. Com $p = 3$ o modelo AR localiza bem a frequência central (≈ 250 Hz), mas representa toda a banda como um único pico largo — não captura a extensão plana do espectro. À medida que a ordem cresce ($p = 16, 32, 64$), o modelo passa a preencher a região de banda larga (≈ 100 a 400 Hz) com um número crescente de picos, representando melhor a porção plana do espectro, porém ao custo de muitas oscilações (picos espúrios). Isso ilustra o compromisso central dos métodos AR: modelos de baixa ordem são bons para sinais de banda estreita (poucos picos agudos), mas representam mal espectros de banda larga.

5 Conclusão

Os doze exercícios dos Blocos 5, 6 e 8 foram resolvidos e executados em GNU Octave, com todos os gráficos e valores numéricos gerados pelos próprios códigos. Os resultados reproduzem as observações esperadas do Capítulo 5 de Semmlow & Griffel: os métodos AR (Yule–Walker, Burg e covariância modificada) são superiores ao método clássico de Fourier para *resolver senoides de banda estreita em ruído*, especialmente com segmentos curtos e SNR baixo, desde que a ordem do modelo seja escolhida com cuidado — ordem baixa demais funde componentes próximos, ordem alta demais introduz picos espúrios. Já para espectros de *banda larga*, os métodos clássicos (Welch) representam melhor a forma e a largura das componentes, enquanto o AR tende a aproximá-las por séries de picos.

A Função sig_noise.m (fornecida com o livro)

```

1 function [waveform_noise, time, waveform, snr_out] = sig_noise(freqsin, snr, N)
2 % [waveform_noise, time, waveform, snr_out] = sig_noise(freqsin, snr, npts);
3 % Function to generate test data Generates sinusoids in noise.
4 % Inputs
5 %   freqsin           is a vector specifying the frequency of sinusoid(s)
6 %                       assuming a sample frequency of 1 KHz
7 %   One sinusoid of amplitude 1 is generated for each entry
8 %   snr               is a vector the SNR values in db of the associated sinusoid
9 %                       if snr is a scalar it is used for all frequencies
10 %   npts              number of points in the array
11 % Outputs
12 %   waveform_noise    is the output vector containing sinusoids and noise
13 %   waveform          is the output containing only sinusoids (no noise)
14 %   time              is the time vector useful in plotting the waveform
15 %                       i.e., plot(time, waveform)
16 %
17 %
18 fs = 1000;           % Assume a sampling freq of 1 kHz
19 Ts = 1/fs;
20 time = (0:(N-1))*Ts;
21 noise = randn(1,N);   % Generate noise and calculate RMS value
22 rms_noise = sqrt(mean(noise.^2));
23 if length(snr) < length(freqsin) && length(snr) == 1 % Check SNR vector length
24     snr(2:length(freqsin)) = snr(1); % If a scalar, use this value for all freq.
25 elseif length(snr) < length(freqsin)
26     disp('Error: not enough SNR values')
27     waveform_noise = rms_noise;
28     return
29 end
30 %
31 for i = 1:length(freqsin)
32     freq_scale = freqsin(i) * 2 * pi/fs;
33     x = (1:N) * freq_scale;
34     snr_n = 10^(snr(i)/20); % Convert from dB
35     A = snr_n * rms_noise * 1.414; % Determine gain for appropriate SNR
36     if i == 1
37         component = sin(x) * A;
38         waveform = component;
39         rms_sig(i) = sqrt(mean(waveform.^2));
40     else
41         component = sin(x) * A;
42         rms_sig(i) = sqrt(mean(component.^2));
43         waveform = waveform + component;
44     end
45     snr_out(i) = 20 * log10(rms_sig(i)/rms_noise); % Confirm SNR
46 end
47 waveform_noise = waveform + noise;

```

B Implementação própria de pmcov.m (covariância modificada)

```

1 function [Pxx, f] = pmcov(x, p, nfft, fs)
2 % PMCOV Estimativa de densidade espectral de potencia pelo metodo AR de
3 % COVARIANCIA MODIFICADA (forward-backward least squares).
4 %
5 % [Pxx, f] = pmcov(x, p, nfft, fs)
6 %
7 % Reimplementacao compativel com a funcao pmcov da Signal Processing
8 % Toolbox do MATLAB, necessaria porque o pacote "signal" do Octave nao
9 % fornece pmcov/pcov. O metodo estima os coeficientes AR minimizando a
10 % soma dos erros de predicao para frente (forward) e para tras (backward),
11 % que e exatamente o metodo da covariancia modificada (Marple/Kay).
12 %
13 % Entradas:
14 % x - vetor do sinal
15 % p - ordem do modelo AR
16 % nfft - numero de pontos da FFT (resolucao em frequencia)
17 % fs - frequencia de amostragem (Hz)
18 %
19 % Saidas:
20 % Pxx - densidade espectral de potencia (unilateral)
21 % f - vetor de frequencias (Hz), de 0 a fs/2
22 %
23 % Referencia: Semmlow & Griffel, "Biosignal and Medical Image
24 % Processing", 3a ed., Cap. 5 (metodos parametricos AR).
25
26 x = x(:); % garante vetor coluna
27 N = length(x);
28 if nargin < 3 || isempty(nfft), nfft = 256; end
29 if nargin < 4 || isempty(fs), fs = 1; end
30
31 % ----- Monta as equacoes de predicao forward e backward -----
32 % Numero de equacoes de cada tipo
33 Ne = N - p;
34
35 % Matriz de dados (forward + backward) e vetor alvo
36 Phi = zeros(2*Ne, p);
37 y = zeros(2*Ne, 1);
38
39 % Equacoes forward:  $x(n) \sim \sum_{k=1..p} c_k x(n-k)$ ,  $n = p+1..N$ 
40 for i = 1:Ne
41     n = p + i;
42     Phi(i, :) = x(n-1:-1:n-p).'; % [x(n-1) x(n-2) ... x(n-p)]
43     y(i) = x(n);
44 end
45
46 % Equacoes backward:  $x(n) \sim \sum_{k=1..p} c_k x(n+k)$ ,  $n = 1..N-p$ 
47 for i = 1:Ne
48     n = i;
49     Phi(Ne+i, :) = x(n+1:n+p).'; % [x(n+1) x(n+2) ... x(n+p)]
50     y(Ne+i) = x(n);
51 end
52
53 % ----- Solucao por minimos quadrados -----
54 c = Phi \ y; % coeficientes preditores
55 a = [1; -c]; % polinomio AR:  $A(z) = 1 - \sum c_k z^{-k}$ 
56
57 % Variancia do ruido de excitacao (erro medio quadratico forward-backward)
58 res = y - Phi*c;
59 e = (res' * res) / length(res);
60
61 % ----- Densidade espectral de potencia a partir dos coeficientes AR -----
62 A = fft(a, nfft);
63 Pxx_full = e ./ (fs * abs(A).^2); % PSD bilateral
64
65 % Converte para unilateral (0 .. fs/2)
66 Nhalf = floor(nfft/2) + 1;
67 Pxx = Pxx_full(1:Nhalf);
68 if rem(nfft,2) % nfft impar: nao dobrar o ultimo bin
69     Pxx(2:end) = 2*Pxx(2:end);
70 else % nfft par: nao dobrar DC nem Nyquist
71     Pxx(2:end-1) = 2*Pxx(2:end-1);
72 end
73
74 f = (0:Nhalf-1).' * (fs/nfft);

```

75

end